KOMPUTASI BERBASIS JARINGAN

TUGAS SOCKET PROGRAMMING

SIMULASI LOAD BALANCING
STUDI KASUS ROUND ROBIN
DAN LEAST REQUESTS



Oleh:

Muhammad Umar Mansyur / 2605190500

Batari Wahyu Pangesti / 26051905021

UNIVERSITAS NEGERI SURABAYA

FAKULTAS TEKNIK

S2 INFORMATIKA

1. Pendahuluan

1.1 Latar Belakang

Dalam sistem terdistribusi, satu server sering tidak cukup untuk melayani seluruh permintaan pengguna. Ketika jumlah permintaan meningkat, waktu respons memburuk dan risiko kegagalan bertambah. Penyeimbang beban (load balancer) menjadi instrumen penting untuk membagi permintaan ke beberapa server pekerja (worker) sehingga beban terdistribusi, tidak ada worker yang dibiarkan menganggur sementara worker lain kelebihan beban, dan kegagalan satu worker dapat ditoleransi.

Studi ini membangun miniatur sistem distributed computing dengan socket programming Python dalam arsitektur client-server. Sistem terdiri atas satu client, satu broker sebagai pengatur lalu lintas permintaan, dan tiga worker sebagai pelaksana komputasi. Client mengirim permintaan (request) ke broker melalui TCP socket; broker memilih satu worker berdasarkan algoritma load balancing, meneruskan permintaan, lalu mengembalikan hasil dari worker kepada client. Seluruh komunikasi memakai protokol JSON satu baris yang diakhiri newline (\n) karena TCP merupakan aliran byte dan tidak mempertahankan batas pesan.

Untuk menguji perilaku distribusi, simulasi dikondisikan dengan dua algoritma yang dibandingkan secara langsung. Pendekatan pertama adalah pemerataan jumlah permintaan yang telah dilayani (least requests): broker memilih worker dengan jumlah request yang sudah selesai dilayani paling sedikit, dengan aturan tambahan ID terkecil bila terjadi seri. Pendekatan kedua adalah Round Robin: broker memilih worker secara bergiliran $1 \rightarrow 2 \rightarrow 3 \rightarrow 1 \ldots$ berdasarkan counter internal, tanpa melihat beban worker. Setiap worker menyediakan dua jenis aplikasi, yaitu SHORT yang cepat (echo) dan LONG yang berat (perhitungan bilangan prima atau faktorial), sehingga perbedaan biaya komputasi antar permintaan dapat diamati. Hasil pemodelan ini diharapkan memberi gambaran empiris mengenai cara kedua algoritma mendistribusikan permintaan, termasuk keterbatasannya ketika satu permintaan LONG jauh lebih berat daripada satu permintaan SHORT.

1.2 Tujuan

Laporan ini bertujuan untuk mengimplementasikan komunikasi client-server melalui socket, membuktikan bahwa broker benar-benar meneruskan permintaan kepada tiga worker yang berbeda, mengimplementasikan dan menguji dua algoritma

alokasi (pemerataan jumlah request dan Round Robin) secara terpisah, serta membuktikan bahwa worker dapat membedakan aplikasi LONG dan SHORT dan mencatat aktivitasnya melalui log.

1.3 Manfaat

Secara teoretis, eksperimen memperkuat pemahaman perbedaan antara penyeimbangan berbasis status (least requests) dan penyeimbangan buta-beban (Round Robin). Secara metodologis, protokol JSON-line, mekanisme framing newline, dan log aktivitas mendukung pengujian yang dapat direproduksi. Secara praktis, studi memberi ilustrasi perancangan broker sebagai single entry point, dampak pemilihan algoritma terhadap pemerataan, serta kebutuhan failover ketika satu worker mati.

1.4 Metodologi dan Sistematika

Pendekatan yang digunakan adalah implementasi dan pengujian deskriptif terhadap sistem nyata (bukan simulasi matematis): kode Python dijalankan, worker dan broker dihidupkan pada port 5001–5003 dan 5000, kemudian client mengirimkan skenario permintaan dan distribusi per worker dicatat dari respons (field worker_id) dan log broker. Laporan disusun menjadi Pendahuluan, Permasalahan, Pembahasan, Penutup, Daftar Pustaka, dan Lampiran. Seluruh angka distribusi berasal dari eksekusi program; waktu respons diukur pada loopback localhost sehingga bersifat indikatif.

2. Permasalahan

2.1 Deskripsi Kasus 1 – Pemerataan Jumlah Request (Least Requests)

Broker memilih worker yang memiliki jumlah request yang sudah dilayani paling sedikit. Sebagai contoh, jika Worker 1 telah melayani 5 request, Worker 2 melayani 3 request, dan Worker 3 melayani 4 request, maka broker memilih Worker 2. Jika dua worker memiliki jumlah yang sama, berlaku aturan tambahan ID terkecil. Perlu dicatat bahwa algoritma ini lebih tepat disebut least requests atau least served requests, bukan least connections, apabila yang dihitung adalah jumlah request yang telah selesai dilayani, bukan jumlah koneksi aktif. Konsekuensinya, algoritma ini menyeimbangkan jumlah request, bukan waktu eksekusi atau penggunaan CPU: satu request LONG dapat jauh lebih berat daripada satu request SHORT.

2.2 Deskripsi Kasus 2 – Round Robin

Broker memilih worker secara bergiliran berdasarkan counter internal yang dimajukan setiap kali alokasi dilakukan. Untuk enam request berurutan, urutan alokasinya adalah Worker 1 → 2 → 3 → 1 → 2 → 3. Round Robin tidak perlu mengetahui worker mana yang paling sedikit bebannya; broker cukup menyimpan counter. Counter bersifat persisten antar koneksi client, sehingga request berikutnya tetap melanjutkan giliran.

2.3 Rancangan Eksperimen

Arsitektur terdiri atas lima komponen logis: satu client, satu broker, dan tiga worker. Pembagian port disepakati sebagai berikut: broker 5000, Worker 1 di 5001, Worker 2 di 5002, Worker 3 di 5003, seluruhnya pada 127.0.0.1. Setiap worker dapat menjalankan kedua jenis aplikasi; tidak ada worker yang dikhususkan untuk LONG atau SHORT.

Format pesan memakai JSON satu baris diakhiri newline. Contoh request SHORT adalah {"request_id": "REQ-001", "application_id": "SHORT", "payload": "Halo Server"} dan worker mengembalikan payload tersebut apa adanya. Contoh request LONG adalah {"request_id": "REQ-002", "application_id": "LONG", "payload": {"operation": "prime_count", "limit": 100000}} dan worker mengembalikan 9592, yaitu banyaknya bilangan prima yang tidak melebihi 100.000. Operasi LONG lain yang didukung adalah factorial. Setiap respons worker memuat field worker_id sehingga distribusi dapat diverifikasi dari sisi client, selain dari log broker ([BROKER] Selected Worker N) dan log worker ([WORKER-N] Received REQ-xxx).

Skenario pengujian yang dijalankan adalah: (a) 6 request SHORT dengan Round Robin; (b) 6 request SHORT dengan Least Requests; (c) 18 request SHORT dengan Least Requests untuk memeriksa pemerataan; (d) 6 request campuran MIXED (SHORT/LONG bergantian); (e) uji fungsional LONG prime_count 100.000/1.000.000/2.000.000 dan factorial 20; (f) uji failover dengan mematikan Worker 2 lalu mengirim 3 request. Setiap skenario dijalankan dengan broker yang baru dinyalakan agar counter mulai dari nol.

2.4 Pertanyaan Analisis

Apakah client dapat berkomunikasi dengan broker melalui socket? Apakah broker benar-benar meneruskan permintaan kepada tiga worker yang berbeda? Bagaimana kedua algoritma mendistribusikan 6 dan 18 request, dan apakah urutannya sesuai prediksi? Apakah worker membedakan LONG dan SHORT dengan benar, berapa hasil dan waktu responsnya? Apa yang dicatat log broker dan worker, dan apa yang terjadi bila satu worker gagal?

3. Pembahasan

3.1 Landasan Teori dan Metode Implementasi

Socket TCP menyediakan aliran byte andal dua arah, tetapi tidak mempertahankan batas pesan. Oleh karena itu setiap pesan JSON diakhiri newline dan penerima membaca sampai newline diterima (readline). Pola ini disebut framing dan cukup untuk tugas ini. Broker membuka satu socket server untuk client dan, untuk setiap request, membuka satu koneksi baru ke worker terpilih, meneruskan JSON yang sama, menunggu satu baris respons, mencatatnya, lalu meneruskannya ke client.

Aturan Round Robin dirumuskan sebagai wid = (counter mod N) + 1 dengan counter += 1 setiap alokasi dan N = 3. Aturan Least Requests dirumuskan sebagai pilihan worker dengan served terkecil, yaitu argmin(served[wid], wid), dengan served[wid] ditambah satu setiap respons worker berhasil diterima. Seri dipecahkan dengan ID terkecil. Pemilihan dan pencatatan dilindungi lock sehingga aman terhadap client konkuren (broker memakai thread per koneksi). Jika worker terpilih gagal (connection refused/timeout), broker otomatis mencoba worker berikutnya dan mencatatnya di log.

Aplikasi SHORT didefinisikan sebagai echo: worker mengembalikan payload tanpa komputasi berat. Aplikasi LONG didefinisikan sebagai komputasi berat: prime_count memakai Sieve of Eratosthenes dalam O(limit log log limit) dan factorial memakai perkalian eksak. Kebenaran prime_count dapat diperiksa: P(100.000) = 9.592, P(1.000.000) = 78.498, dan P(2.000.000) = 148.933. Batas yang diterima adalah limit 0–2.000.000 dan n faktorial 0–1000 untuk mencegah transfer raksasa.

3.2 Hasil Eksperimen

Tabel 1. Distribusi request per worker (hasil eksekusi nyata).

Skenario	Worker 1	Worker 2	Worker 3
Round Robin, 6× SHORT	2 (REQ-001, 004)	2 (REQ-002, 005)	2 (REQ-003, 006)
Least Requests, 6× SHORT	2 (REQ-001, 004)	2 (REQ-002, 005)	2 (REQ-003, 006)
Least Requests, 18× SHORT	6	6	6
Round Robin, 6× MIXED	2 (1 SHORT + 1 LONG)	2 (1 SHORT + 1 LONG)	2 (1 SHORT + 1 LONG)
Failover, 3× SHORT (W2 mati)	2	0 (down)	1

Pada kedua strategi dengan broker baru (counter nol), enam request pertama menghasilkan urutan identik 1, 2, 3, 1, 2, 3 dan pembagian 2-2-2. Hal ini wajar: Least Requests mulai dari served {0, 0, 0} sehingga pilihan pertama jatuh ke ID terkecil, kedua ke sisa terkecil, dan seterusnya, yang untuk permintaan sekuensial berperilaku sama dengan Round Robin. Delapan belas request menghasilkan 6-6-6 pada kedua strategi. Skenario MIXED membuktikan kedua jenis aplikasi terdistribusi merata (masing-masing worker menerima 1 SHORT dan 1 LONG), sekaligus membuktikan tidak ada worker yang dikhususkan untuk satu aplikasi. Skenario failover membuktikan ketahanan: ketika Worker 2 dimatikan, broker mencatat “Worker 2 failed (Connection refused), trying next...” lalu meneruskan ke Worker 3/1, sehingga ketiga request tetap berhasil dengan distribusi 2-0-1.

Tabel 2. Kebenaran fungsional dan waktu respons (loopback localhost, indikatif).

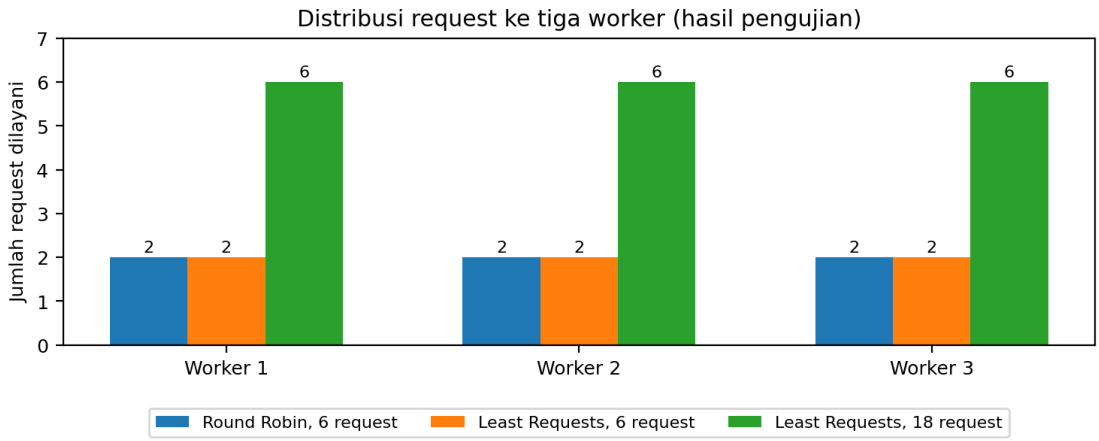
Permintaan	Hasil	Waktu respons
SHORT echo “Halo Server”	“Halo Server” (tepat)	±1 ms

LONG prime_count limit 100.000	9592 (benar)	±1–3 ms
LONG prime_count limit 1.000.000	78.498 (benar)	±9 ms
LONG prime_count limit 2.000.000	148.933 (benar)	±13 ms
LONG factorial n = 20	2432902008176640000 (benar)	±1 ms

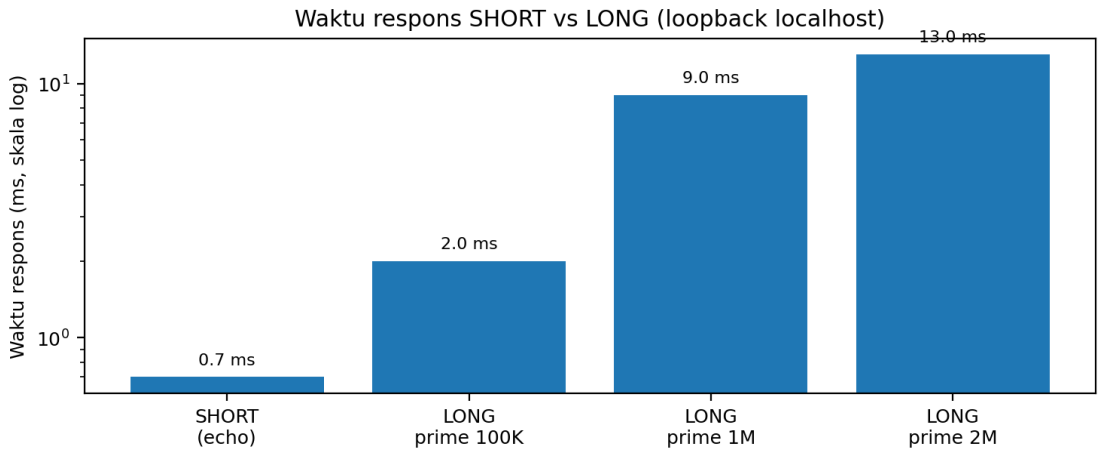
Seluruh hasil komputasi sesuai nilai yang diharapkan. Waktu respons menunjukkan LONG secara konsisten lebih berat daripada SHORT (prime 1M sekitar 13 kali lipat echo pada pengujian ini), meskipun angka absolutnya kecil karena sieve dalam Python masih cepat untuk limit tersebut dan komunikasi loopback hampir tanpa latensi jaringan. Perbedaan akan membesar pada limit lebih besar, mesin lebih lambat, atau beban konkuren.

3.3 Visualisasi Distribusi

Gambar 1 merangkum Tabel 1: setiap skenario normal menghasilkan batang yang sama tinggi (2-2-2 untuk 6 request, 6-6-6 untuk 18 request), sedangkan skenario failover menunjukkan batang Worker 2 nol. Gambar 2 membandingkan waktu respons dalam skala logaritmik: SHORT tercepat, kemudian prime 100K, 1M, dan 2M yang meningkat seiring limit, sesuai kompleksitas sieve.



Gambar 1. Distribusi request ke tiga worker. Skenario normal terbagi rata; skenario failover (W2 mati) hanya memakai W1 dan W3.



Gambar 2. Waktu respons SHORT vs LONG pada loopback localhost (skala log). LONG meningkat seiring limit prima.

3.4 Interpretasi Pendekatan A (Pemerataan Jumlah Request)

Pendekatan ini terbukti menyeimbangkan jumlah request yang sudah dilayani: 2-2-2 untuk 6 request dan 6-6-6 untuk 18 request. Log broker memperlihatkan pola "Request REQ-xxx received – Application – Strategy: LEAST_REQUESTS – Selected Worker N – Response received – Response sent to Client" untuk setiap request, dan log worker memperlihatkan pasangan "Received – Processing – Response sent" pada worker yang benar. Kelebihannya adalah adaptif terhadap riwayat: worker yang tertinggal akan diprioritaskan. Keterbatasannya bersifat fundamental dan sesuai penegasan tugas: yang diseimbangkan adalah jumlah request, bukan beban CPU. Satu LONG prime 2M tetap dihitung "satu" sama seperti satu echo, sehingga worker yang kebetulan menerima lebih banyak LONG menanggung waktu CPU lebih besar.

3.5 Interpretasi Pendekatan B (Round Robin)

Round Robin terbukti menghasilkan urutan deterministik 1, 2, 3, 1, 2, 3 untuk enam request dan pembagian 2-2-2, dengan counter yang persisten antar koneksi client (request MIXED melanjutkan giliran dari request sebelumnya). Kelebihannya adalah sederhana, murah (O(1) tanpa status per worker selain counter), dan adil untuk request homogen seperti 6× SHORT. Keterbatasannya sama: buta terhadap biaya permintaan. Pada skenario MIXED, setiap worker tetap menerima tepat satu LONG, tetapi itu kebetulan karena pola genap; pada pola ganjil

atau LONG beruntun, Round Robin dapat menumpuk LONG pada worker yang sama tanpa menyadarinya. Round Robin juga tidak mempertimbangkan kegagalan historis kecuali melalui mekanisme failover.

3.6 Sintesis dan Refleksi Kritis

Untuk permintaan sekuensial yang semuanya berhasil, kedua algoritma pada pengujian ini tidak dapat dibedakan dari distribusinya (keduanya 2-2-2 dan 6-6-6). Perbedaan baru muncul bila (a) served awal tidak seimbang, (b) ada kegagalan worker, atau (c) permintaan datang konkuren sehingga status served berubah antar pemilihan. Failover menjadi pembeda praktis yang teramati: Least Requests memilih ulang berdasarkan served terkini, sedangkan Round Robin melanjutkan putaran dari counter. Keduanya membutuhkan mekanisme framing newline, thread per koneksi, dan timeout agar satu worker lambat tidak menggantung broker. Yang terpenting, pemerataan jumlah bukan pemerataan beban: evaluasi lanjutan memerlukan penimbangan biaya (weighted) atau pengukuran beban aktif bila LONG dan SHORT dicampur dalam produksi.

4. Penutup

4.1 Kesimpulan

Keempat hal yang dinilai tugas terbukti. Pertama, komunikasi client-server melalui socket berfungsi: client mengirim JSON-line ke broker dan menerima respons yang memuat worker_id. Kedua, distribusi request terbukti: 6 request menghasilkan 2-2-2 dan 18 request menghasilkan 6-6-6 pada kedua algoritma, dengan urutan 1, 2, 3, 1, 2, 3 yang tercatat di log broker. Ketiga, dua algoritma dapat dipilih dan diuji terpisah melalui flag --strategy (round_robin atau least_requests, dengan alias least_connections). Keempat, pemrosesan aplikasi dan logging terbukti: SHORT mengembalikan echo, LONG prime_count 100.000 mengembalikan 9592, dan setiap sisi mencatat aktivitas sesuai format yang diminta. Temuan kunci adalah pemerataan jumlah request tidak menjamin pemerataan beban komputasi ketika LONG jauh lebih berat daripada SHORT.

4.2 Keterbatasan dan Saran

Pengujian dilakukan sekuensial pada loopback localhost sehingga waktu respons kecil dan belum mencerminkan latensi jaringan, konkurensi tinggi, atau

worker di mesin berbeda. Sieve masih cepat untuk limit sampai 2 juta; pembeda LONG/SHORT akan lebih tajam dengan limit lebih besar atau payload yang mensimulasikan kerja I/O. Saran pengembangan adalah menambah health check periodik, bobot biaya per application_id (weighted round robin / weighted least requests), batas timeout dan retry yang eksplisit, serta metrik beban CPU per worker agar pemilihan mempertimbangkan biaya aktual, bukan hanya hitungan request.

Daftar Pustaka

Kurose, J. F. dan Ross, K. W. Computer Networking: A Top-Down Approach. 8th ed. Pearson, 2021.

Python Software Foundation. socket — Low-level networking interface. Dokumentasi Python 3.10. https://docs.python.org/3/library/socket.html.

Python Software Foundation. json — JSON encoder and decoder. Dokumentasi Python 3.10. https://docs.python.org/3/library/json.html.

Cardellini, V., Casalicchio, E., Colajanni, M., dan Yu, P. S. The state of the art in locally distributed Web-server systems. ACM Computing Surveys, 34(2):263–111, 2002.